Test Case Correctness & Coverage: Design Analysis

TestAtlas Team

October 18, 2026

CONTENTS

Test Case Correctness & Coverage

Validating LLM-Generated Functional Test Cases in TestAtlas

1. Executive Summary

2. What Has Been Built

3. Problem Statement

4. Proposed Approaches (As Discussed)

4.1 Idea 1 — Compare generated test cases against the existing test suite

4.2 Idea 2 — Generate user stories instead of test cases, and build requirements under each story

4.3 Idea 3 — Compare existing user stories against the code to measure coverage

5. Recommended Approach — STAR Analysis

Situation

Task

Action

Track A — Grounding (Problem A)

Track B — Coverage (Problem B)

Result

6. Cross-Cutting Challenges

7. Recommended Next Step

CONFIDENTIAL

Test Case Correctness & Coverage

Validating LLM-Generated Functional Test Cases in TestAtlas

CONFIDENTIAL

1. Executive Summary

TestAtlas analyzes a codebase, builds a knowledge graph of it, and uses that graph to generate functional test cases automatically. This has worked well as a way to get a first draft of test coverage for a codebase with little or no documentation. It has one open problem: there is no reliable way to say whether a generated test case is actually *true* — that it describes something the application really does — or whether, as a set, the generated test cases actually cover what matters to the business.

This document lays out that problem precisely, records the three approaches proposed during design discussion, evaluates each on its own merits, and proposes a two-track strategy that treats “is this test case accurate” and “do we have the right test cases” as separate problems requiring separate solutions. It closes with the practical challenges that apply regardless of which techniques are adopted, and a recommended order of investigation.

No implementation has been started. This is a design-stage document intended to align on approach before any of it is built.

CONFIDENTIAL

2. What Has Been Built

TestAtlas is a FastAPI + SQLite platform, backed by a vanilla JS front end, that performs static analysis on a source repository and turns it into something a QA or product team can act on.

Knowledge graph construction. The repository is parsed into a graph of nodes (files, classes, functions) and edges (calls, imports, references). This graph is the source of truth for everything downstream.

Module clustering. Nodes are grouped into *modules* — today, this grouping follows the repository's own package/folder structure (e.g. `components.cart`, `services.auth`). Each module is scored with metrics including PageRank, to identify central, high-impact code.

Findings and risk flags. The graph is scanned for structural risk signals (bottlenecks, highly-coupled nodes, orphaned code) and surfaced as findings.

Gap analysis. When a user uploads existing documentation (architecture docs, process docs), TestAtlas compares the documentation's claims against the code and reports three categories: missing implementation, undocumented capability, and mismatch. This is the one feature that already compares an external, human-authored source of truth against the code — it is directly relevant to the recommendation in Section 5.

LLM-based test case generation. For a chosen module, an LLM is given the module's code, the real UI strings/labels extracted from it, and any uploaded documents, and asked to produce functional test cases in Azure DevOps shape: title, description, priority, preconditions, and steps that pair an action with an expected result. Each module's generation is a separate, individually-triggered, metered LLM call — nothing runs in bulk, and nothing is auto-approved. The product explicitly frames these as **first drafts for human review**, not verified facts, and includes a code-traceability reference on each generated case pointing back to the module and functions it was derived from.

Export. Generated test cases export to CSV (Azure DevOps import format), Markdown, and JSON.

This system produces useful first-draft test coverage quickly. The question this document addresses is what happens next: how much should anyone trust a given generated test case, and how do we know the *set* of generated test cases is complete in a way that matters to the business, not just to the code's internal structure.

CONFIDENTIAL

3. Problem Statement

Functional test cases are generated by reasoning over **code relations** — which functions call which, what strings and labels appear in the UI layer, how modules are structured. This produces fluent, plausible-sounding test cases. It does not produce anything that comes with a built-in way to check it.

This splits into two genuinely different problems that are easy to conflate:

Problem A — Grounding ("is this test case true"). A generated step claims the application shows a specific message, enables a specific button, or follows a specific flow. There is no automatic way today to confirm the claim matches reality rather than a plausible invention. Unlike a unit test, a generated functional test case is prose — it has no built-in pass/fail signal, and it is explicitly not runnable code.

Problem B — Coverage ("do we have the right test cases"). Modules are an artifact of how the repository happens to be organized into packages, not of how a real user experiences the product. A single business workflow such as "checkout" is very likely to be split, in the code, across several unrelated modules — login, search, cart, payment, address. Generating test cases per-module, bottom-up, will always produce a set of tests shaped like the codebase's internal structure, not like a user's journey through the product. Without correcting for this, it is not possible to say "checkout is 80% tested" — only "these five unrelated modules each have some tests."

Any solution needs to treat these as two tracks. A technique that makes individual test cases more trustworthy does nothing to tell you whether the right scenarios were generated in the first place, and vice versa.

CONFIDENTIAL

4. Proposed Approaches (As Discussed)

Three approaches were proposed and discussed before this document was produced. They are recorded here in the order raised, each evaluated on how far it goes toward solving Problem A, Problem B, or both.

4.1 Idea 1 — Compare generated test cases against the existing test suite

Description. Where a repository already has an automated test suite (unit/integration tests), diff the generated functional test cases against it, on the theory that overlap or agreement is evidence of correctness.

How it addresses the problem. Primarily Problem A (grounding) — it treats the existing suite as a partial oracle.

Pros	Cons
Reuses an artifact that likely already exists in the repository, at no extra authoring cost.	Existing tests and generated functional test cases operate at different altitudes — a unit test asserts `returns 401 for invalid credentials`; a generated test case narrates `user sees an error message and is not logged in`. These do not line up for a direct text diff.
Where the suite does assert specific strings or status codes, those are checkable facts that can be cross-referenced against claims in a generated step.	Only validates the narrow slice of claims that happen to overlap with something the existing suite checks — most functional claims (flow, visible behavior) will have no corresponding assertion to compare against.
Does not require running anything — a textual/structural comparison is enough for the narrow use described above.	Says nothing about correctness where no existing test exists for that code path, which is disproportionately the repositories this product is most useful for (low test maturity).

Verdict. Useful as a narrow, low-cost fact-check layered on top of another technique. Not viable as a standalone correctness signal.

CONFIDENTIAL

4.2 Idea 2 — Generate user stories instead of test cases, and build requirements under each story

Description. Rather than generating test cases per code module, generate user stories first (bottom-up, from the same graph), then generate test cases underneath each story.

How it addresses the problem. Aimed at Problem B (coverage/relevance).

The granularity mismatch, identified during discussion. A real user story such as "Checkout" is, from the code's point of view, implemented across several independently-clustered modules: login, search, cart, payment, address. A module-clustering algorithm run bottom-up over the import/call graph has no signal that tells it these five modules compose into one story — it will reproduce the module graph under a different name, not the real story graph.

Pros	Cons
Would, if it worked, give test cases a business-relevant anchor instead of a technical one.	The core mechanism — deriving stories from the same bottom-up module clustering already used for code structure — inherits exactly the problem it's meant to fix. Modules are a package/folder artifact; stories are a user-journey artifact; one does not derive the other.
No new external input required, in principle.	Would likely need an entirely different signal (e.g. tracing invocation sequences across modules) to produce journey-shaped groupings, which is a materially harder graph problem than today's clustering, and was not fully specified.

Verdict. Correctly motivated, but as stated, does not solve the problem it targets — it relocates the granularity mismatch rather than resolving it. (Section 5.2 proposes a mechanism, derived from this same discussion, that targets the same goal without the same flaw.)

4.3 Idea 3 — Compare existing user stories against the code to measure coverage

Description. Instead of generating stories from code, take user stories from an external, human-authored source (the same kind of document already ingested by the Gap Analysis feature) and measure, per story: what percentage of its steps are detectably implemented in the code, and whether existing test cases cover those steps. This produces a 2x2 view per story: developed vs. not, tested vs. not.

CONFIDENTIAL

How it addresses the problem. Directly targets Problem B, and — because it anchors to real, human-defined scope — avoids idea 2's granularity mismatch entirely. A story's boundary is defined by the document, not by the code, so "checkout spans five modules" is simply the expected shape of the answer, not a clustering failure.

Pros	Cons
Sidesteps the granularity mismatch by construction — the human-authored story is the unit of measure, and code is traced *into* it, many-to-one, which is the natural direction for a journey that spans modules.	Requires the customer to already have written user stories to anchor against. Many repositories — especially the under-documented ones this product targets — will not have this input available.
Reuses and extends an existing, already-shipped capability (Gap Analysis already compares documents against code).	Introduces a new, unsolved mapping problem: given a story's steps, which code modules/functions does each step actually correspond to? Nothing in the current system performs this mapping; it would need to be built.
Produces a coverage number that is legible to a non-technical stakeholder ("Checkout is 70% implemented, 40% tested") in a way a raw module list is not.	The mapping step above is likely to have the same grounding problem as test case generation itself — an LLM asked "which modules cover step 3 of this story" can be wrong with no obvious signal that it is wrong.

Verdict. The strongest of the three as a coverage strategy, with one hard dependency (a written story) and one hard unsolved sub-problem (the step-to-code mapping) that would need to be designed before this is viable end to end.

CONFIDENTIAL

5. Recommended Approach — STAR Analysis

Situation

Generated test cases come with no built-in way to check either (a) whether an individual test case is accurate, or (b) whether the generated set, as a whole, covers what a business stakeholder would consider “done.” Idea 3 is the strongest available fix for (b), but it depends on an external document that will not always exist, and leaves the step-to-code mapping unsolved. None of the three ideas discussed addresses (a) on its own.

Task

Define a strategy that: (1) improves confidence in individual generated test cases without requiring a runnable environment for the target repository, since most repositories analyzed by this product cannot be built, seeded, and executed on demand; and (2) produces a business-relevant coverage view that does not strictly require the customer to already have written user stories, while still taking advantage of them when they do exist.

Action

Treat this as two independent tracks, built and evaluated separately.

TRACK A — GROUNDING (PROBLEM A)

Rather than one technique, layer cheap, mostly-deterministic checks first, and reserve expensive or environment-dependent checks for where they are actually available.

1. **Deterministic citation checking (do this first).** Require every generated step to cite the specific graph node it was derived from — an extension of the traceability reference already present in exports today, from *display-only* to *enforced and verified*. After generation, a plain, non-LLM check confirms the cited node exists and that any claimed string/label actually appears in that node’s extracted source. This is close to free (no extra model call), fully deterministic, and directly catches the most common and most damaging failure mode: inventing a UI element or message that does not exist.

Limitation to carry forward: this bounds hallucination of *existence*, not of *behavior*. A step can cite a real function and still describe its behavior incorrectly.

CONFIDENTIAL

2. **Self-consistency sampling.** Generate the same module's test cases more than once — a different temperature, or a second model — and treat claims that recur across independent runs as more likely grounded, and claims that vary as more likely invented. This needs no oracle at all, and fits the "first draft" framing already built into the product.

Limitation to carry forward: consistency is not correctness — a model can be consistently wrong the same way every time (a common training-data assumption, such as assuming a "forgot password" flow exists when it does not). It also multiplies the per-module metered cost by the number of samples taken, for a signal that item 3 below can often reach more cheaply.

3. **A separate critic pass, not resampling.** A second LLM call, with a distinct prompt framed as an adversarial check rather than a generation task, is given the code and the already-generated test case, and asked to find claims unsupported by the code, citing the specific contradicting or supporting line. A distinct prompt frame tends to catch more than resampling the same generation prompt (item 2), because it is not anchored to the same assumptions that produced the original claim — for that reason, this is the better default escalation after citation checking, with self-consistency sampling held in reserve for claims specific enough that a second independent generation is the more informative check.

Limitation to carry forward: a critic built on the same model family shares some of the same blind spots as the generator, and simply producing more flagged items does not by itself solve who reviews them.

4. **Execution-based verification and coverage cross-referencing — opportunistic, not default.** Where a target repository happens to be runnable, or already has a working automated test suite with usable code coverage data, both are strong, close-to-ground-truth signals (did the described step actually happen; has this code path ever actually executed). Both should be designed as an *optional enhancement* layered on top of Track A, not a dependency — the majority of repositories this product analyzes will not offer a runnable environment, and executing arbitrary third-party code changes the product's security and sandboxing posture in a way that static analysis today does not.

5. **Treat the output as a calibrated confidence signal, not a verdict.** None of the above produces a binary true/false. The practical target is a per-test-case or per-claim confidence signal a reviewer can triage by, and — once any edit/accept/reject affordance exists on generated test cases — a feedback loop that calibrates that confidence against real reviewer behavior over time. This also argues for a small, team-maintained benchmark of hand-verified repositories, run periodically against the generation pipeline, to catch regressions whenever the generation prompt or underlying model changes — the same role the project's existing automated test suite plays for its own code.

CONFIDENTIAL

TRACK B — COVERAGE (PROBLEM B)

Idea 3 is the right mechanism when a written story exists; it should be built as described, extending Gap Analysis to operate at story granularity instead of (or alongside) its current document-vs-code comparison. For the far more common case where no story has been written, the graph already contains a usable signal that was not fully explored in the original discussion:

Cluster by entry point / request path, not by package. Modules fragment "Checkout" because today's clustering runs over the *import/package* graph — an engineering-organization artifact. If, instead, real entry points (HTTP route handlers, top-level page/view components, CLI commands) are identified and the *outbound call chain* from each one is traced, the resulting grouping is shaped like "everything that runs for this request" — which tracks a real user journey far more closely than package structure does, because it is derived from actual invocation order rather than static import relationships. This requires no external document and works on every repository, including fully undocumented ones — precisely where Idea 3 cannot help.

This is not a complete substitute for Idea 3; a single entry point is often narrower than a full business journey (one route is "add to cart," not the whole of checkout), so some aggregation of related entry-point chains into a higher-level story is still likely necessary. That remaining aggregation step is a meaningfully easier problem than today's package-based clustering, because invocation order gives much stronger sequencing signal than raw import relations do.

Result

Pursued together, these two tracks produce a system that can say something closer to: *"Checkout (story, from your own documentation, or inferred from its entry points) is 70% implemented, has 12 generated test cases, 9 of which cite verified, existing code and show no contradictions from the critic pass, and 3 of which are flagged for review."* That is a materially more defensible claim than "here are some auto-generated test cases," without requiring a runnable environment, an existing test suite, or a pre-written user story as a hard dependency for every repository.

CONFIDENTIAL

6. Cross-Cutting Challenges

These apply regardless of which specific techniques from Section 5 are prioritized first.

No runnable environment for most repositories. The product's core promise is zero-setup analysis of a source snapshot. This structurally rules out or severely limits every execution-based technique (dynamic step verification, coverage cross-referencing) for a large share of real usage — these should remain optional enhancements, never a dependency.

Cost compounds. Any technique that adds LLM calls (critic pass, self-consistency sampling) multiplies the per-module metered cost that the product already surfaces to users as a visible concern. Cheap, deterministic checks (citation verification) should be exhausted before reaching for additional model calls.

The output will be a confidence distribution, not a verdict. Nearly every viable technique here produces "probably fine," not "verified true." The open product question is not only which technique to build, but how a confidence signal is surfaced to a reviewer in a way that stays useful rather than becoming noise that gets ignored — this deserves explicit design attention, not just a numeric score bolted onto the existing UI.

Grounding and coverage are independent failures. A perfectly grounded test case for an irrelevant scenario, and a perfectly relevant test case with one invented step, are both "wrong" in different ways. Solving one does not solve the other, and both tracks in Section 5 need to proceed in parallel rather than sequentially if either is to be useful on its own.

Entry-point detection is new engineering, not a reuse of existing parsing. Reliable entry-point detection (HTTP routes, page components, CLI commands) is a different detection problem per framework/language — closer in shape to the existing repository parsing work, but not a direct extension of it, and would need its own per-framework coverage plan.

CONFIDENTIAL

7. Recommended Next Step

Before building anything: validate the cheapest, highest-leverage piece first — deterministic citation checking (Section 5, Track A, item 1) — since it requires no new model calls, directly targets the most damaging failure mode (invented facts), and can be measured against the existing hand-verified understanding of a small number of repositories without any further design work. Everything else in this document should wait on what that first measurement shows.

CONFIDENTIAL